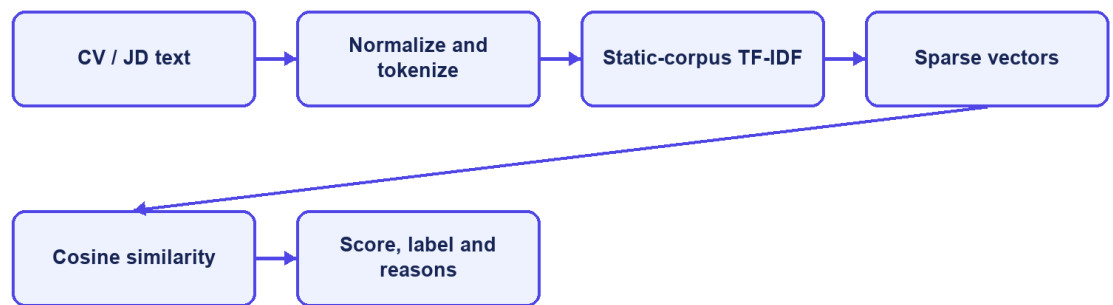


For non-negative TF-IDF vectors, cosine similarity usually lies between 0 and 1. A larger value indicates a greater proportion of shared weighted terms. Length normalization is valuable for CV–JD comparison because a long CV should not receive a higher score merely because it contains more words. TF-IDF-weighted cosine distance has also been used as an efficient and reproducible document-alignment method [3].

Cosine similarity remains a lexical measure. If a JD uses “container orchestration” while a CV only uses “Kubernetes,” the vectors may not overlap unless the vocabulary or normalization layer establishes that relationship. Conversely, shared generic terms can increase similarity without proving that mandatory requirements are satisfied. CareerFit therefore treats similarity as one component of a broader workflow that also includes structured fields, validation, labels, reasons, user feedback, and policy conditions.

TF-IDF and cosine pipeline



CareerFit IT AutoPilot — thesis diagram

Figure 2.2. TF-IDF vectorization and cosine-similarity pipeline

2.4 Matching, Recommendation, and Ranking

Matching and recommendation can be formalized as ranking tasks. Given a query representation q and a candidate set D , the system computes a relevance score $s(q,d)$ for each $d \in D$ and orders the results by decreasing score. For candidate-to-job matching, q may be a CV vector and D the active job vectors. For recruiter-side discovery, q may be the selected JD and D the available candidate or CV vectors. For profile-based recommendation, q represents the candidate’s desired role, skills, location, seniority, and other preferences rather than one uploaded CV.

This separation matters because a CV describes past experience, while a preference profile describes what the candidate wants. A candidate may have Java experience but look for a Go position, so CV-based matching and preference-based recommendation can produce different but valid lists. Keeping the workflows separate also makes evaluation clearer because the labels, candidate sets, and user goals of one task should not be assumed to fit the other task.

A raw similarity value is not automatically a calibrated probability of hiring success. Mapping it to a percentage or label is a presentation and business-rule decision, not a statistical guarantee. Thresholds such as Low, Medium, High, or Potential should be documented, tested at boundary values, and kept distinct from application status. Stable secondary sorting is also necessary when multiple items have equal scores so that repeated requests do not appear inconsistent.

Recent resume–job matching research increasingly uses dense encoders and contrastive learning. ConFit v2, for example, improves resume–job representations through hypothetical resumes and hard-negative mining [7]. Such approaches can capture semantic relations beyond exact terms, but they require training data, model governance, and an evaluation design appropriate to the domain. CareerFit intentionally begins with a transparent lexical baseline whose calculations can be reproduced and explained, while treating semantic or hybrid retrieval as future work rather than an implemented result.

2.5 Relevance Feedback and the Rocchio Method

2.5.1 Relevance Feedback

Relevance feedback uses judgments about retrieved items to modify a query or profile representation. Instead of assuming that the original text completely expresses the user’s information need, the system observes which results are marked relevant or non-relevant. Explicit feedback is especially useful in recruitment because the importance of related skills may be known to the candidate or recruiter but omitted from the original CV, profile, or JD.

The classic Rocchio method updates a query vector by combining the original query with the centroids of relevant and non-relevant document vectors [4]. Let q_0 be the original vector, D_r the set of relevant feedback documents, and D_{nr} the set of non-relevant documents. A common form is:

$$q_m = \alpha q_0 + (\beta / |D_r|) \sum (d \in D_r) d - (\gamma / |D_{nr}|) \sum (d \in D_{nr}) d.$$

The parameters α , β , and γ control retention of the original representation, attraction toward relevant examples, and movement away from non-relevant examples. They are hyperparameters rather than universal constants. Positive and

negative feedback may also be weighted differently because an explicit rejection can be ambiguous: a user may reject a job because of location, timing, or salary even when its technical content is relevant.

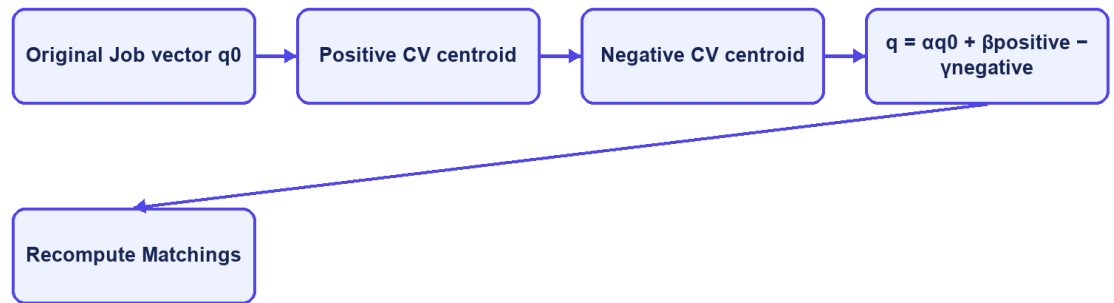
2.5.2 Feedback Semantics in CareerFit

CareerFit distinguishes feedback such as `GOOD_MATCH`, `POTENTIAL`, `BAD_MATCH`, and `NOT_INTERESTED`. These labels should not be collapsed without a documented mapping. `GOOD_MATCH` provides a strong positive relevance signal. `POTENTIAL` may represent partial or transferable fit rather than direct relevance. `BAD_MATCH` is a negative judgment about the match. `NOT_INTERESTED` can reflect personal intent and may be weaker evidence about technical similarity. The implementation chapter must state exactly how each event affects the learned vector and whether role or channel changes its weight.

A reliable update process keeps the original base vector and rebuilds the learned vector from the current feedback history. Updating the previously learned vector again and again can cause cumulative drift: the same recomputation may change the output even when there is no new feedback. In CareerFit, the same base vector, feedback set, and parameters should always produce the same learned vector. This repeatable behavior is important for retries, scheduled jobs, debugging, and audits.

Feedback learning should be evaluated by comparing rankings before and after a defined feedback event in a controlled setup. The experiment records the baseline ranking, adds feedback, rebuilds the representation, and measures the effect on separate holdout items. If the same positive item is used both as feedback and as the only proof of improvement, the result is circular. Chapter 4 therefore separates feedback examples from holdout evaluation and clearly labels synthetic scenarios.

Rocchio relevance feedback



CareerFit IT AutoPilot — thesis diagram

Figure 2.3. Rocchio relevance-feedback vector update

2.6 Evaluation Metrics for Ranked Results

Ranking quality cannot be described well by ordinary classification accuracy because users see an ordered list and usually inspect only the first few items. Evaluation therefore needs relevance judgments and metrics that consider the cutoff K , the positions of relevant results, and graded relevance when it is available.

2.6.1 Precision@K, Recall@K, and HitRate@K

Precision@K is the proportion of the top K results that are relevant. Recall@K is the proportion of all known relevant items that appear in the top K . HitRate@K records whether at least one relevant item appears in the first K positions. Precision emphasizes the usefulness of the visible list, recall emphasizes coverage, and HitRate provides a simple task-success indicator. The metrics answer different questions and should not be substituted for one another.

2.6.2 Mean Reciprocal Rank

For a query, reciprocal rank is $1/r$, where r is the rank of the first relevant result; it is zero if no relevant result is retrieved. Mean Reciprocal Rank (MRR) averages this value across queries. MRR strongly rewards placing the first relevant result near the top but ignores the ordering of additional relevant items. It is therefore appropriate when finding one useful job or candidate is the primary objective, but it should be accompanied by a broader top- K metric when multiple relevant results matter.

2.6.3 Discounted Cumulative Gain

Discounted Cumulative Gain (DCG) supports graded relevance and discounts useful items that occur at lower ranks. Normalized DCG divides the observed DCG by the ideal ordering for the same relevance judgments, producing nDCG values that are comparable across queries. Järvelin and Kekäläinen introduced cumulated-gain measures to credit systems for ranking highly relevant documents before less relevant ones [5]. One common formulation is:

$$DCG@K = \sum_{i=1..K} (2^{rel_i} - 1) / \log_2(i + 1), \quad nDCG@K = DCG@K / IDCG@K.$$

Metric values are meaningful only when the relevance labels, candidate set, cutoff K, and calculation method are documented. A large improvement in a synthetic scenario shows behavior only in that scenario; it does not prove hiring quality in production. Chapter 4 therefore reports the data source, baseline, holdout logic, repeated-run behavior, and threats to validity together with the metric values.

2.7 Human-in-the-Loop and Explainable Automation

2.7.1 Levels of Human Involvement

Human-in-the-Loop is not a single interface feature. Human involvement can occur during data validation, model configuration, review of recommendations, approval of actions, exception handling, and post-action feedback. NIST describes human–AI configurations as ranging from manual decision making through systems that provide an additional opinion or defer to an expert, to more autonomous operation [9]. The appropriate configuration depends on the consequences and failure modes of the task.

Recruitment decisions can strongly affect people, so a similarity engine should help users set priorities instead of acting as the final decision-maker. CareerFit separates input processing, decision support, actions, learning, and audits. The matching engine provides evidence; the AutoFit policy checks whether an action is allowed; users can review or confirm important actions; feedback changes later rankings; and audit records support investigation. This design does not remove bias, but it makes the control points and system state clearer.

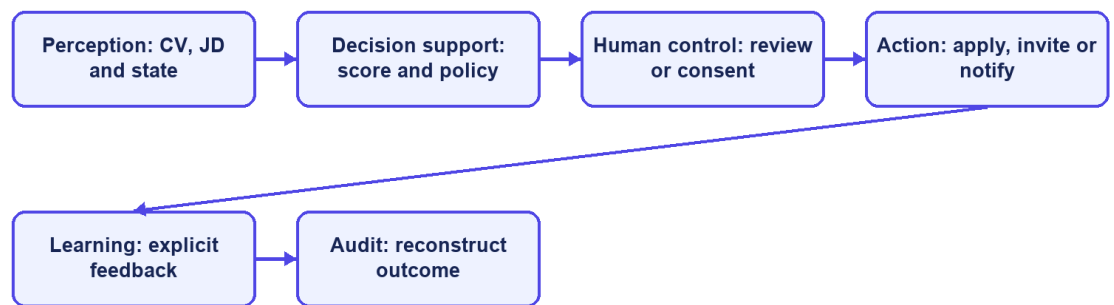
2.7.2 Explainability and Auditability

Explainability describes why a result or action was produced. Auditability asks whether the inputs, rules, actor, channel, time, and outcome can be checked later. A list of shared skills can explain lexical similarity, but it does not prove that the whole model or decision is fair. In the same way, an audit log records what happened but does not prove that the policy was correct.

Research on algorithmic hiring warns that claims about reducing bias and improving performance must be checked against real practices, data, and organizational context [10]. Explanations can help users understand a result, but they should be based on the features that the system actually uses. Research on explainable recommendation also separates scoring from the user-facing reasons shown with a result [14]. CareerFit therefore uses match reasons and policy outcomes that can be checked instead of unsupported natural-language claims about candidate suitability.

The NIST AI Risk Management Framework groups AI risk work into governance, mapping, measurement, and management [9]. CareerFit is not a complete implementation of this framework. However, its design follows several practical ideas: state the scope and limitations, keep people responsible for important actions, measure behavior with defined evidence, and keep records for later review.

Human-in-the-Loop cycle



CareerFit IT AutoPilot — thesis diagram

Figure 2.4. Human-in-the-Loop control cycle in CareerFit

2.8 Web Architecture, Security, and Audit Foundations

2.8.1 Client–Server and REST

CareerFit uses a web client and a server-side application connected through HTTP APIs. The client is responsible for presentation, navigation, local interaction state, and invoking authorized operations. The backend enforces authentication, authorization, validation, business rules, transactions, persistence, and background processing. This separation prevents browser-side presentation logic from becoming the authority for protected actions.

REST is an architectural style characterized by constraints including client–server separation, stateless interaction, cacheability where appropriate, a uniform interface, layered components, and optional code-on-demand [11]. A REST API models resources and transfers representations of their state. In practice, merely using JSON over HTTP does not guarantee that every REST constraint is satisfied; Chapter 3 therefore describes CareerFit as a REST-oriented API and documents its actual endpoint and state-transition behavior.

2.8.2 Authentication, Authorization, and Action Tokens

Authentication establishes the identity associated with a request, while authorization determines whether that identity may perform an operation. Role-Based Access Control assigns permissions according to roles such as Candidate, Recruiter, or Administrator, but ownership checks are still required; for example, one candidate must not access another candidate’s private CV merely because both share the same role.

JSON Web Token is a compact, URL-safe format for transferring claims that can be signed or message-authentication-code protected and may include registered claims such as subject, audience, expiration time, issued-at time, and token identifier [12]. A valid signature alone is not sufficient: the application must validate the expected algorithm and relevant claims and must apply its own account and authorization rules. Short-lived access tokens and purpose-specific action tokens address different workflows and should not be treated as interchangeable credentials.

An actionable email link may be opened by the intended user, forwarded by mistake, or visited by a mail-security scanner. CareerFit uses a two-step flow to reduce this risk. The GET request checks the high-entropy token and shows a confirmation page without changing recruitment data. A deliberate POST request performs the action and marks the token as redeemed. Only the SHA-256 token hash is stored, and status and expiry checks prevent normal reuse. Production

deployment should still add rate limiting, origin controls, secret rotation, and mail-delivery monitoring.

2.8.3 Audit Logging

Audit logging records security-relevant and business-relevant events so that operations can be monitored and investigated. NIST guidance emphasizes a managed logging process that includes generation, transmission, storage, access, analysis, and disposal rather than treating logs as incidental text files [13]. For recruitment automation, useful fields include the actor, action, target, source channel, relevant policy or score snapshot, result, and timestamp. Sensitive CV content and credentials should not be copied into logs without a clear need.

An append-oriented audit history supports traceability, but database permissions, retention, integrity, and access control determine whether that history is trustworthy. Operational logs used for debugging and domain audit records used to explain a business action may have different schemas and retention requirements. Chapter 2 defines these responsibilities at the design level, and Chapter 3 describes the mechanisms actually implemented in CareerFit.

2.9 Related Work and Research Gap

Prior work can be grouped into lexical retrieval, learned resume–job representation, explainable recommendation, and human-centered governance. The classical vector-space and relevance-feedback literature provides transparent mathematical foundations [1], [4]. Contemporary resume–job matching methods such as ConFit v2 use trained dense representations and hard-negative strategies to improve semantic matching [7]. Real-world observational research comparing human and LLM resume ratings found only minor correlation between the two, indicating that automated and human judgments should not be treated as interchangeable [8]. Explainable recommendation research seeks to accompany scores with understandable, preferably grounded reasons [14].

CareerFit does not attempt to outperform these research systems on a shared benchmark. Its engineering gap is different: connecting an inspectable lexical baseline and explicit relevance feedback to an end-to-end recruitment workflow with role-based interfaces, policy gating, actionable email, and auditable state changes. The resulting contribution is system integration and controlled automation within an IT-focused academic prototype.

Table 2.1. Positioning of CareerFit against major approach categories

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
----------	----------	--	--------------------

Approach	Strength	Limitation relative to CareerFit scope	CareerFit position
Lexical TF-IDF/cosine	Transparent, efficient, reproducible	Limited semantic equivalence	Implemented baseline with normalization, reasons, and validation
Dense resume–job encoders	Captures semantic relations	Requires training data, governance, and benchmark validation	Future hybrid/semantic extension; not claimed as implemented
General job portal	Strong search, publication, and application UX	May not expose scoring, feedback, or policy internals	Combines portal workflows with inspectable matching and automation
Fully automated decision pipeline	Reduces manual steps	Higher control, accountability, and error-propagation risk	Policy-gated HITL actions with confirmation and audit
Explainable recommender	Provides user-facing reasons	Explanation may be unfaithful if not grounded	Uses reasons tied to processed fields and lexical evidence

The comparison shows that CareerFit deliberately accepts the semantic limitations of a lexical baseline in exchange for inspectability and implementation control. Its research value must therefore be assessed through reproducible behavior, workflow integration, and clearly stated limitations rather than through unsupported claims of general AI superiority.

2.10 Theoretical Background Summary

This chapter presented the main ideas used by CareerFit. TF-IDF and cosine similarity provide an inspectable lexical baseline, while Rocchio uses explicit feedback to adjust ranking. Ranking metrics measure ordered results, and Human-in-the-Loop policies keep similarity evidence separate from recruitment actions. Chapters 1 and 2 turn these ideas into system requirements and design decisions.

2.11 System Architecture

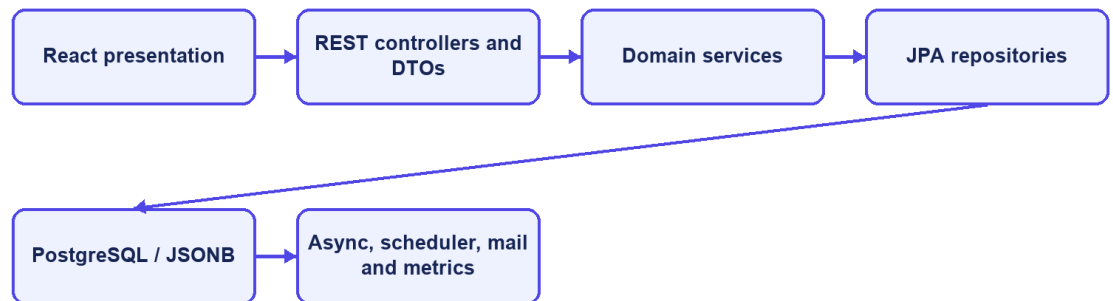
CareerFit is implemented as a modular monolith. The frontend is a separate React application, while the backend is one Spring Boot deployable application organized into domain packages. This is not a microservice architecture: modules share one process, one primary PostgreSQL database, common security and exception handling, and direct in-process service calls. The modular monolith reduces deployment and distributed-transaction complexity while preserving boundaries that can support later extraction if scale or ownership requires it.

The presentation layer contains React pages, reusable components, route guards, API mapping, and query state. It calls the backend over HTTP and must not be trusted to enforce protected business rules. The API layer contains controllers and DTO validation. The service layer owns use-case orchestration, transactions, scoring, policy decisions, and ownership checks. Repositories map domain entities

to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

Modular-monolith architecture



CareerFit IT AutoPilot — thesis diagram

Figure 2.5. CareerFit container and component architecture

2.12 Module Design

Table 2.2. Backend modules and responsibilities

Module	Main responsibility	Representative services
Auth and Security	Registration, password login, JWT validation, account resolution, and role rules	AuthService, JwtService, security filters
Candidate, CV, and Skill	Profile/portfolio, CV draft/review, storage, OCR cleanup, validation, and skill suggestions	CandidateProfileService, CvIngestionService, PdfExtractionService, SkillService
Job and Employer	Public catalogue, company onboarding, JD quality, draft/publish, urgent/deadline, and recruiter-owned lifecycle	JobService, EmployerService, QualityValidationService

Module	Main responsibility	Representative services
Matching	TF-IDF/cosine score, labels/reasons, skill-transfer Potential, batch matching, filters, and ranking	ScoringService, SkillTransferService, MatchingService, MatchingQueryService
Recommendation	Candidate-profile Job ordering and similar-Job retrieval	RecommendationService
Application and Talent	Apply/withdraw, applicants, invitations, status changes, Talent Pool, and CV bookmarks	ApplicationService, RecruiterTalentService
Feedback	Feedback validation, Rocchio update, and recomputation signaling	FeedbackService, RocchioService
Automation	Per-account policy, run-now, pause/resume, auto-apply, reminders, and scheduler	AutomationPolicyService, AutoApplyService, AutomationScheduler
Notification	Policy guard, delivery log, SMTP/no-op mail, and signed email actions	NotificationPolicyGuard, NotificationEmailService, EmailActionService
Analytics, Admin, and Audit	Live analytics, moderation, monitoring, audit persistence, and common API behavior	Analytics services, administrative services, audit/common components

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

2.13 Data Design

2.13.1 Core Identity and Recruitment Data

`user_account` stores identity, role, activation, verification, language, and password hash. A Candidate owns a profile, multiple CVs, reviewable CV sections, extracted skills, portfolio links, and portfolio projects. A Recruiter owns an employer profile and Jobs. A Job stores its JD, structured recruitment fields, draft/publish state, urgent flag, application deadline, application count, source details, and vector data.

`matching` is the unique association between one CV and one Job. It stores the direct score, label, separate Potential flag and explanation, reasons, and recomputation state. `application` links a Candidate and Job and may reference the CV and Matching used at that time. `feedback` stores a judgment about a Matching. `recruiter_cv_bookmark` keeps a Recruiter's saved Candidate/CV for an owned Job, while `recommendation_interaction` records events such as viewed, skipped, applied, saved, not interested, or show similar.

2.13.2 Automation, Communication, and Operational Data

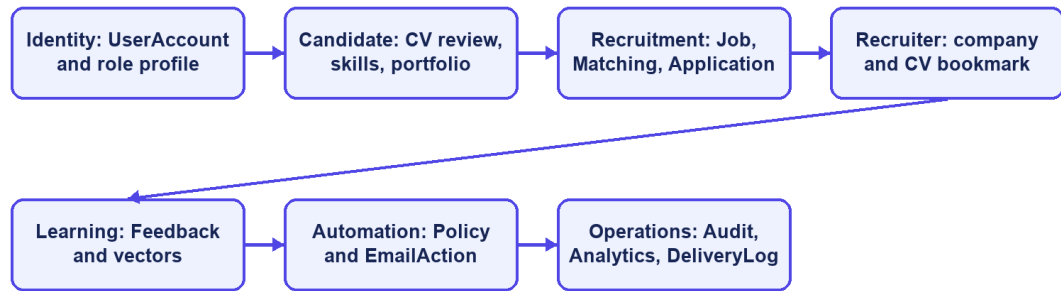
automation_policy stores one policy per account, including enable/pause state, thresholds, quotas, cooldowns, quiet hours, category guards, and notification preferences. email_action stores expiring one-time actions using a token hash. Notification delivery records support deduplication and policy decisions. audit_log records the actor, action, target, result, source channel, request details, and structured metadata. Analytics tables support aggregate dashboards without changing Matching records.

Table 2.3. Key integrity constraints

Constraint	Purpose
Unique normalized user email	Prevent duplicate identities that differ only by case
One Candidate/Recruiter profile and one policy per account	Preserve role ownership and account-level automation settings
Partial unique default CV per Candidate	Ensure at most one active default CV
Unique Matching per CV and Job	Prevent duplicate direct scores for the same pair
Unique Application per Candidate and Job	Prevent duplicate applications or invitations
Unique Recruiter bookmark per Job and Candidate	Prevent duplicate Talent Pool bookmarks
Unique feedback per Matching and actor	Keep one current judgment per actor
Check constraints on roles, states, labels, salary, and source channels	Reject invalid domain values at the database boundary
Indexes for active/urgent/popular Jobs, deadlines, ownership, and ranking	Support server-side catalogue, reminder, and matching queries

Flyway migrations are the authoritative schema history. Hibernate uses ddl-auto=validate, so the application validates entity-to-schema compatibility instead of silently generating a different database. Indexes support common access paths such as active jobs, recruiter jobs, CV ownership, matching by score, applications by candidate or job, feedback history, tokens, and audit chronology.

Data domains and relationships



CareerFit IT AutoPilot — thesis diagram

Figure 2.6. CareerFit logical entity relationships

2.14 Security, Failure, and Consistency Design

2.14.1 Authentication and Authorization

The backend is stateless at the HTTP session layer. A JWT authentication filter runs before username/password authentication processing, and a user-ID resolution filter runs after JWT validation. Public access is explicitly limited to authentication entry points, selected job/employer/analytics GET endpoints, health/metrics/API documentation, and email-action redemption. Candidate, Recruiter, and Administrator endpoint groups require corresponding roles. Automation requires authentication, with service logic responsible for policy ownership.

The security configuration disables CSRF because the application uses stateless bearer authentication, configures a CORS origin allow-list, denies framing, applies a content-security policy, and configures HSTS. These headers do not replace TLS termination in the deployed environment. Public Swagger and Prometheus access are acceptable for local development but should be restricted in a production deployment.

2.14.2 Failure Handling

CV processing represents long-running work with statuses rather than holding a browser request open until every score completes. Extraction or validation failure records a failure state and reason. CV and Job matching tasks are submitted through an after-commit executor so background work cannot read uncommitted rows. Batch scoring isolates individual pair failures, and scheduled recomputation keeps unsuccessful rows marked for later review instead of falsely marking them complete.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

2.14.3 Identified Design Risks

Table 2.4. Design risks and required treatment

Risk	Current control	Remaining treatment
Unauthorized cross-account access	URL role rules and service ownership checks	Maintain negative integration tests for every identifier-based operation
Duplicate applications or matchings	Service checks and unique constraints	Convert database conflicts into stable API errors
Replayed email action	Pending/redeemed state and expiry	Implemented hashed storage, GET confirmation, and POST execution; retain replay and expiry tests
Link scanner triggers action	GET is non-mutating and displays confirmation	Add rate limiting and deployment-specific origin controls
Scheduler/configuration drift	Scheduler timing reads from app.scheduler properties	Maintain configuration and schedule-boundary tests
Sensitive data in logs	Structured audit metadata and DTO boundaries	Review retention, redaction, access, and exported evidence
Lexical scoring bias or omission	Reasons, validation, feedback, and HITL	Evaluate representative data; never treat score as hiring probability
Local file-storage loss or exposure	Configured path and Docker volume	Define backup, encryption, malware scanning, and retention before production

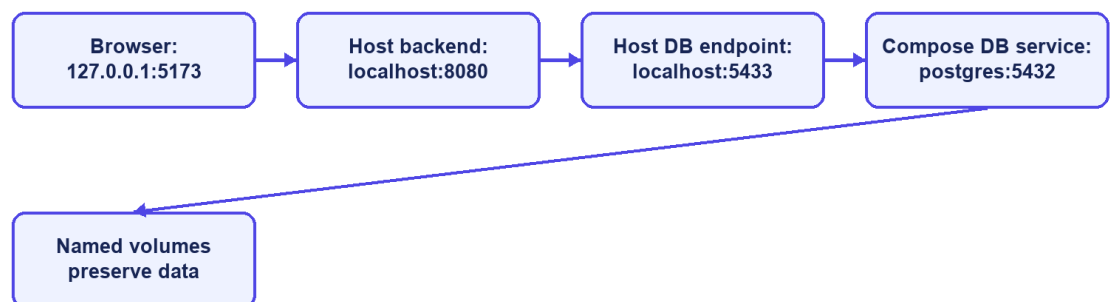
2.15 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, depends on PostgreSQL health, and mounts a named volume for CV storage. The React/Vite frontend is normally run separately on port 5173 during development and calls the backend API. Environment variables override database credentials, JWT secret, application base URL, CORS origins, mail settings, OCR executable and languages, and storage path. Tesseract with Vietnamese and English language data is included in the backend container path described by the project Docker configuration.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

Local deployment addresses



CareerFit IT AutoPilot — thesis diagram

Figure 2.7. Local and containerized deployment topology

2.16 Chapter Summary

This chapter connected the theoretical foundations to the CareerFit solution design. TF-IDF, cosine similarity, Rocchio feedback, ranking metrics, and Human-in-the-Loop principles explain the core decision-support behavior. The architecture, modules, data model, security boundaries, consistency rules, and deployment topology define how those ideas are separated into implementable components.

CHAPTER 3. SYSTEM IMPLEMENTATION

3.1 Implementation Overview

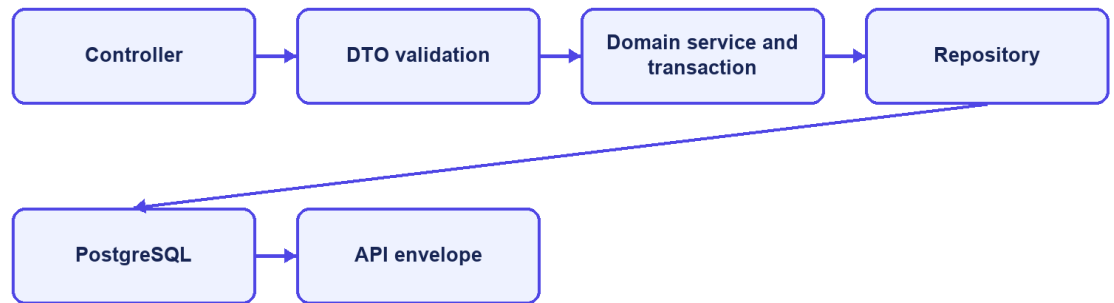
CareerFit is implemented as two application projects and a shared runtime environment. The backend is a Java 21 application based on Spring Boot 3.2.5. The frontend is a React 18 single-page application written in TypeScript and built with Vite. PostgreSQL is the transactional database, Flyway owns schema migration, and Docker Compose provides the reproducible local database and optional backend container. The implementation follows the modular-monolith design established in Chapter 2: domain packages share one backend process while retaining controllers, services, repositories, entities, and DTOs for each functional area.

Table 3.1. Main implementation technologies

Layer	Technology	Implementation purpose
Backend language/runtime	Java 21	Domain logic, API, background processing, and integrations
Application framework	Spring Boot 3.2.5	Dependency injection, web MVC, configuration, scheduling, and application lifecycle
Security	Spring Security and JWT 0.12.5	Stateless bearer authentication, role authorization, and JWT handling
Persistence	Spring Data JPA, PostgreSQL 16, Flyway	Entity persistence, queries, constraints, indexes, and versioned migrations
Document processing	PDFBox 3.0.2, Apache POI, Tesseract CLI	PDF/DOCX extraction and OCR fallback for images or scanned PDFs
API documentation	springdoc-openapi 2.5.0	OpenAPI documents and Swagger UI
Monitoring	Spring Actuator, Micrometer Prometheus registry	Health and HTTP/application metrics
Frontend	React 18.3, TypeScript 5.9, React Router 7, React Query 5	Role-based pages, typed API integration, navigation, and server-state queries
Visualization and UI	Recharts and Lucide React	Charts and interface icons
Build and test	Maven, npm/Vite, JUnit, Testcontainers, Playwright	Compilation, automated backend tests, integration environment, and browser E2E tests

The backend package root is `com.careerfit.backend`. Domain packages include `auth`, `candidate`, `cv`, `skill`, `job`, `employer`, `matching`, `recommendation`, `application`, `feedback`, `automation`, `notification`, `analytics`, `admin`, `audit`, and `settings`. Shared responses, exceptions, text utilities, validation, security filters, and configuration are placed in common or configuration packages.

Backend request path



CareerFit IT AutoPilot — thesis diagram

Figure 3.1. Backend module structure and request flow

3.2 Authentication and Authorization Implementation

3.2.1 Login and JWT Processing

AuthController exposes registration, email/password login, and current-account operations. AuthService checks account state and credentials and delegates JWT creation to JwtService. Passwords are stored with BCryptPasswordEncoder. A successful login returns an access token and a user DTO with the ID, email, full name, role, and verification state. Passwordless login was removed; signed one-time tokens remain only for explicit email actions.

Each authenticated request passes through JwtAuthenticationFilter, a OncePerRequestFilter. The filter requires the Bearer prefix when an Authorization header is present, validates token structure and signature, extracts subject and role, rejects roles outside Candidate, Recruiter, and Administrator, and reloads the account from the database. Reloading prevents a cryptographically valid token from continuing to authorize a deleted or suspended account. The filter then creates a Spring Security authentication object with the corresponding ROLE_* authority.

UserIdResolutionFilter runs after JWT authentication. It resolves the persisted account ID from the authenticated email and stores the UUID as the userId request attribute. Controllers can therefore pass a stable identifier to services without accepting a user ID supplied by the browser. The implementation currently performs a user lookup in both filters; this is straightforward but creates duplicate database reads that could be consolidated later.

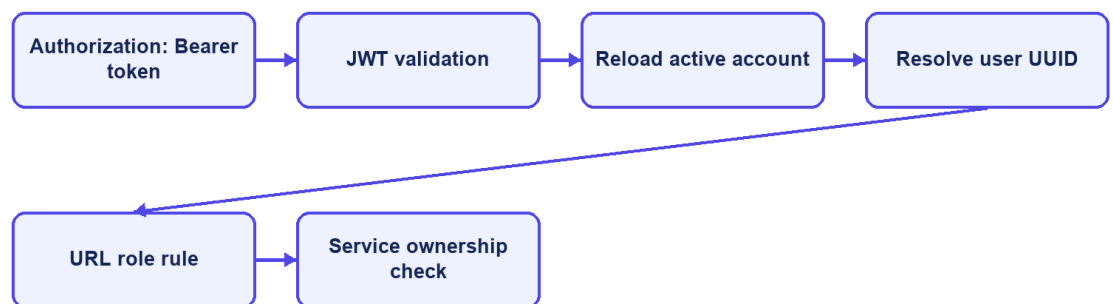
3.2.2 URL Rules and Ownership Checks

SecurityConfig uses a stateless session policy. Public access is limited to selected authentication operations, public job/employer/analytics GET routes, similar-job retrieval, health/metrics/OpenAPI routes, and email-action redemption. Candidate routes cover CV, matching, candidate profile, and applications. Recruiter routes cover owned jobs, candidate discovery, application review, employer profile, and export. Administrative routes require the Administrator role. Automation routes require authentication, with policy ownership resolved in services.

Role checks are necessary but insufficient for resources addressed by UUID. ApplicationService, for example, verifies that a requested CV belongs to the authenticated Candidate and that a Recruiter owns the Job before listing applicants, changing status, or inviting a candidate. The same pattern is required across identifier-based services. Errors are returned through shared security and application error writers so that 401 unauthenticated and 403 forbidden responses remain distinguishable.

The configuration disables CSRF because bearer tokens are used instead of a server-side browser session. CORS origins are loaded from application configuration. Frame denial, Content Security Policy, and HSTS headers are configured. HSTS is effective only when the application is served through HTTPS, so the local HTTP runtime does not itself demonstrate transport security.

JWT request processing



CareerFit IT AutoPilot — thesis diagram

Figure 3.2. JWT authentication and authorization boundaries

3.3 CV Ingestion and Validation

3.3.1 Upload and Manual Creation

CvController provides upload, manual creation, manual drafts, CV listing, details, processing status, review read/update, review confirmation, default selection, and deletion. CvIngestionService coordinates the workflow. Uploaded content is extracted and converted into reviewable sections. Manual entry can be saved as a DRAFT. The Candidate can correct sections and confirm them before vectorization and Job scoring start.

Table 3.2. CV processing states

State	Meaning	Typical transition
DRAFT	Manual content is saved but not submitted for scoring	Draft edit → review/confirm
UPLOADED	Uploaded metadata and source are accepted	Upload → validation/extraction
VALIDATING	File, content, and extraction checks are running	Valid source → review; hard error → failed
REVIEW_REQUIRED	Extracted/manual sections are ready for Candidate review	Edit sections → confirm
PROCESSING	Confirmed content is normalized and vectorized	Vector saved → asynchronous scoring
SCORING_DONE	The confirmed CV vector is available and Matchings can be queried	Use cards, feedback, applications, and AutoFit
FAILED	Processing cannot continue	Failure reason is stored and audited

3.3.2 File Validation and Extraction

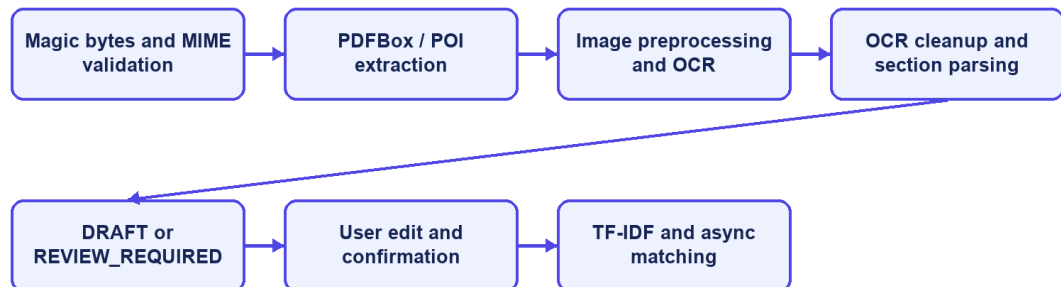
PdfExtractionService accepts PDF, PNG, JPG/JPEG, and DOCX. It checks extension, MIME type, and leading magic bytes. PDFBox extracts embedded PDF text, Apache POI extracts DOCX text, and images use OCR. Scanned pages are preprocessed before Tesseract runs, and OCR cleanup removes common noise before the text is split into review sections. The default OCR languages are vie+eng, with configured page and time limits.

Text shorter than 50 characters after extraction and cleanup is a hard failure. Text from 50 to 199 characters is accepted with a sparse-content warning. Encrypted PDFs are rejected, image dimensions are limited, and temporary OCR files are deleted. The review step also lets the Candidate fix extraction mistakes before the CV is used for scoring. Production use would still need malware scanning and stronger isolation.

QualityValidationService produces structured signals for questionable CV or JD content, such as missing sections, seniority/experience conflicts, salary/content

conflicts, and invalid deadlines. Blocking errors are separated from warnings so users can fix the data instead of losing their work.

CV ingestion and review implementation



CareerFit IT AutoPilot — thesis diagram

Figure 3.3. CV ingestion implementation pipeline

3.4 Text Normalization and TF-IDF Implementation

3.4.1 Text Normalization

TextNormalizationService removes HTML tags, replaces characters outside its English/Vietnamese alphanumeric pattern, lowercases text, splits on whitespace, removes tokens shorter than two characters, and filters a language-specific stop-word set. Language detection is a lightweight heuristic: text containing more than five Vietnamese diacritic characters is classified as Vietnamese; otherwise it is treated as English.

This implementation is deterministic and easy to inspect, but it is not a linguistic word segmenter. Vietnamese multi-syllable skills can be split into independent tokens, and punctuation-containing technology names such as C++, C#, .NET, or Node.js may lose distinguishing characters. These limitations are important when interpreting matching errors and motivate a future domain-aware tokenizer or controlled alias dictionary.

3.4.2 Static Corpus and Vector Construction

TfIdfService builds its IDF map once at application startup from a static seed corpus of representative IT terms. The corpus groups programming languages, frameworks, databases, cloud and DevOps technologies, architecture, testing, security, data/ML, collaboration, seniority, roles, and common Vietnamese IT words. A static corpus prevents scores from shifting whenever a user uploads one CV or adds one Job.

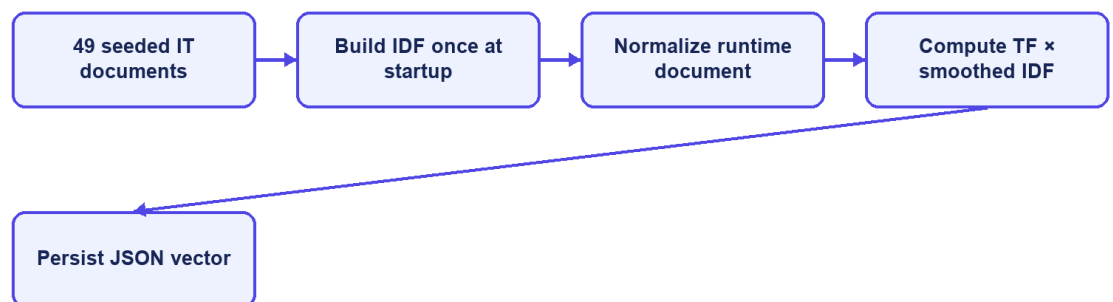
For a token list, term frequency is the token count divided by the total number of tokens. The implemented smoothed IDF is:

$$idf(t) = \log(1 + N / (1 + df(t))).$$

An unknown term receives $\log(1 + N)$, which treats it as rare and informative. The TF-IDF vector is persisted as JSON and reused during scoring. Cosine similarity iterates over the smaller vector for the dot product, computes both Euclidean magnitudes, and returns zero when either vector is empty.

The unknown-term policy has a practical trade-off. It preserves project-specific technology terms absent from the seed corpus, but a rare misspelling can receive the same high IDF treatment. Input validation, alias normalization, corpus versioning, and evaluation on realistic data are therefore necessary before treating the score as stable beyond the academic environment.

Static-corpus TF-IDF



CareerFit IT AutoPilot — thesis diagram

Figure 3.4. Seed-corpus initialization and TF-IDF construction

3.5 Matching and Recommendation Implementation

3.5.1 Scoring Service

ScoringService loads the CV vector from extractedTermsJson. For the Job, it prefers learnedProfileVectorJson when a non-empty learned vector exists; otherwise it uses the original tfidfVectorJson. Cosine similarity becomes the raw score, which is multiplied by 100 and rounded to two decimal places. The raw score is stored with six decimal places.

Table 3.3. Implemented score interpretation

Condition	Stored/displayed result
Direct score below 70	LOW label
Direct score from 70 to below 90	MEDIUM label
Direct score at least 90	HIGH label; Potential is not added
Potential score at least 62 with skill compatibility ≥ 0.50 and family compatibility ≥ 0.55	Potential guard continues
Core target and transferable career evidence exist, with no severe seniority gap	isPotential=true and an explanation is stored
Direct score below 20 and skill compatibility below 0.70	Potential is rejected as weak evidence

The direct-match label uses the configured medium boundary (70) and high boundary (90), producing LOW, MEDIUM, or HIGH. Potential is a separate boolean and does not replace that label. The current Potential model is loaded from a versioned JSON knowledge base and is evaluated separately from cosine similarity.

Direct-match reasons use the highest-weight Job terms that also appear in the CV, with the Job domain added when available. Potential uses normalized aliases, direct skill-transfer links, role-family transfer, shared foundations, and seniority compatibility. Its explanation can list transferable skills and shared foundations. This remains a rule-based explanation, not a learned claim that the Candidate will succeed.

3.5.2 Matching Orchestration and Persistence

After the Candidate confirms the CV review, vectorization is committed and AfterCommitExecutor submits MatchingService.scoreAllJobsForCv. The service reloads the CV, scores eligible active Jobs, and creates or updates the unique Matching row. Reasons and Potential metadata are stored as JSON. The same post-commit boundary is used when a published Job is scored against existing confirmed CVs.

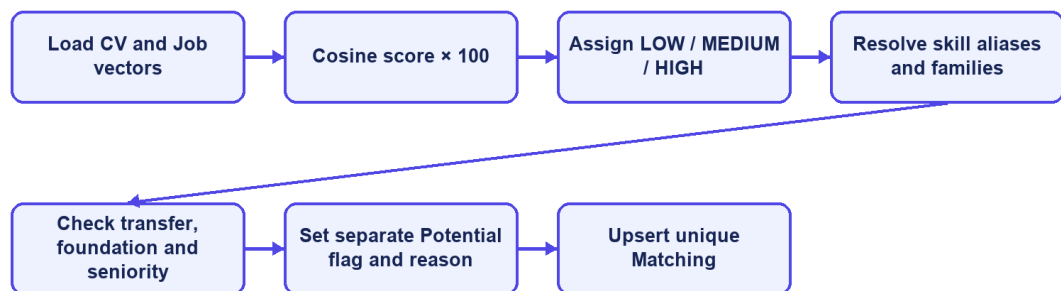
A Recruiter first completes a company profile, then can save a Job as DRAFT, check JD quality, and publish it with the required deadline. Published Jobs can be marked urgent and are scored after commit. MatchingQueryService supports

Candidate and Recruiter views with server-side filters, pagination, sorting, label/Potential filters, and clear empty-state metadata. Portfolio details remain protected by the Candidate's visibility setting and application state.

3.5.3 Recommendation Service

RecommendationService serves personalized Job results and similar-job retrieval. Candidate recommendations use profile/preferences and available matching information, while similar jobs can be retrieved publicly. The API and UI intentionally keep recommendations separate from the persisted application workflow. A recommendation can be viewed, skipped, or used to start an application without becoming an application automatically unless an enabled AutoFit policy separately authorizes that action.

Direct match and Potential assessment



CareerFit IT AutoPilot — thesis diagram

Figure 3.5. Scoring and matching persistence flow

3.6 Feedback Learning with Rocchio

FeedbackService.submitFeedback resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching_id, actor_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD_MATCH, POTENTIAL, and BAD_MATCH trigger post-commit Job-vector learning.

RocchioService locks the Job for update and always loads the original Job TF-IDF vector as q. Positive examples are CV vectors attached to GOOD_MATCH or

POTENTIAL feedback; negative examples come from BAD_MATCH. The service computes term-wise centroids and applies:

$$q_{new} = 1.0q + 0.75 \text{ positive_centroid} - 0.15 \text{ negative_centroid}.$$

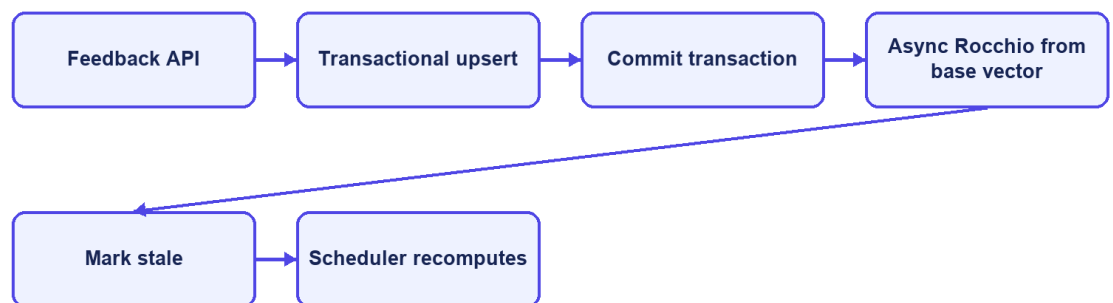
Negative weights are removed rather than persisted. The learned vector is stored in learnedProfileVectorJson. Every Matching associated with the Job is then marked needsRecompute=true. AutomationScheduler.recomputeStaleMatchings rescans these rows every 30 minutes and clears the marker only after successful scoring.

Using the original base vector and the complete feedback history makes recomputation repeatable for the same data and parameters. FeedbackService starts learning after the transaction commits, so Rocchio reads the saved feedback instead of racing the transaction that created it. Job locking prevents two updates from writing separate learned vectors at the same time, and integration tests cover this post-commit behavior.

Table 3.4. Feedback handling

Feedback	Persisted	Rocchio classification	Immediate learning trigger
GOOD_MATCH	Yes	Positive	Yes
POTENTIAL	Yes	Positive	Yes
BAD_MATCH	Yes	Negative	Yes
NOT_INTERESTED	Yes	Neither	No

Feedback implementation



CareerFit IT AutoPilot — thesis diagram

Figure 3.6. Feedback processing and post-commit learning

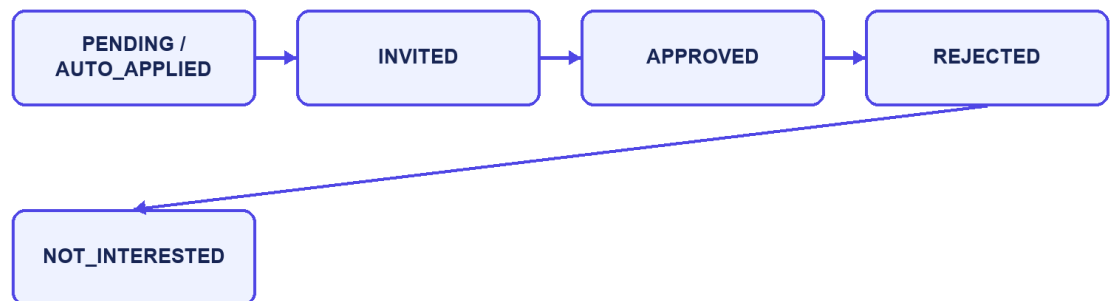
3.7 Application and Recruiter Workflow

`ApplicationService.submit` resolves the authenticated Candidate, requires an active Job before its application deadline, rejects an existing Candidate–Job application, and uses the requested or default confirmed CV. If a Matching exists, it is attached to keep the score context. The service saves an audit event, updates the Job application count through the database rule, and requests Candidate and Recruiter notifications.

A Candidate views owned applications through a paginated API and can filter them by status in the interface. A Recruiter can list applicants only for an owned Job, update valid states, or invite a Candidate. The Talent Pool adds Job-based Candidate discovery, a Potential-only view, and CV bookmarks so a Recruiter can save a profile before inviting the Candidate.

Response DTOs combine application, Candidate, CV, Matching, Job, and bookmark details without exposing persistence entities. Portfolio fields are returned only when the Candidate has enabled visibility and the application state permits it. Server-side pagination, sorting, and filters keep larger Job and Candidate catalogues manageable.

Application state flow



CareerFit IT AutoPilot — thesis diagram

Figure 3.7. Application and invitation state transitions

3.8 AutoFit and Background Processing

3.8.1 Async Executor and Scheduler

`AsyncConfig` creates a bounded `ThreadPoolTaskExecutor`. `AfterCommitExecutor` submits CV and Job matching only after the related transaction commits. Mail sending and Rocchio learning can also leave the request thread, while persisted CV, Matching, email-action, and delivery states make completion and failures observable.

`AutomationScheduler` coordinates matching recomputation, digests, high-match checks, auto-apply, token cleanup, and deadline-related reminders using configured schedules. Per-item error handling prevents one failed record from stopping a full scan. Account policies add pause/resume, thresholds, quotas, cooldowns, quiet hours, and category guards before eligible actions or notifications are executed.

3.8.2 Auto-Apply

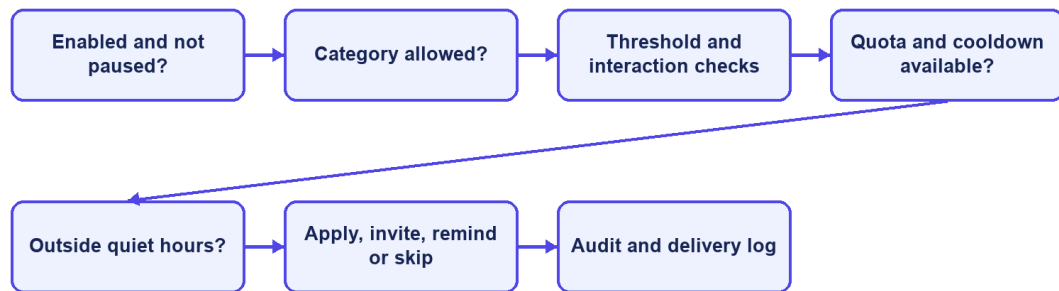
`AutoApplyService.runForPolicy` resolves the policy owner, Candidate, and confirmed default CV. It checks policy enablement and pause state, eligible categories, active Job state, deadline, score threshold, previous interactions, quota, and cooldown before creating an automatic application. Database uniqueness remains the final concurrency guard. Every created or skipped outcome can be audited and followed by a notification decision.

Auto-apply and notification delivery use related but separate checks. Application creation requires consent and application-policy conditions, while email delivery also checks timing, quota, cooldown, category, and deduplication. Keeping the two decisions separate prevents a notification setting from silently becoming permission to submit an application.

3.8.3 Notification Guard and Delivery

`NotificationPolicyGuard` evaluates whether a message is allowed and stores sent, skipped, or failed results in separate transactions. It supports deduplication, quota, cooldown, quiet hours, category guards, and deadline reminders. SMTP is used when enabled, while `NoOpMailService` keeps local development usable without claiming that real email delivery was tested.

Per-account automation guard



CareerFit IT AutoPilot — thesis diagram

Figure 3.8. Scheduler and AutoFit execution boundaries

3.9 Email Action and Audit Implementation

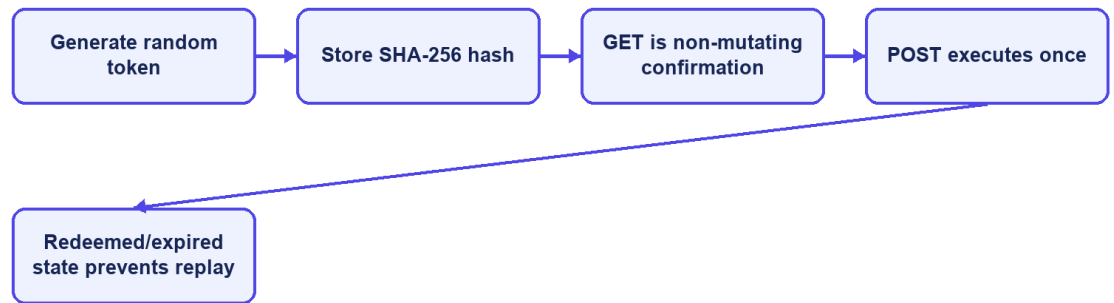
EmailActionService generates a 32-character token derived from UUID text for each supported action and stores it with recipient, optional Matching, type, status, and expiry. Match email actions include GOOD_MATCH, POTENTIAL, NOT_INTERESTED, and view links. Digest messages create actions for listed Matches and unsubscribe intent. Tokens remain valid for 72 hours.

EmailActionController exposes a public confirmation and redemption flow because possession of the high-entropy token is the credential. The raw token is sent to the user but only its SHA-256 hash is persisted. GET validates the token and renders a non-mutating confirmation page; POST performs the supported action, records redemption, and rejects unknown, expired, or already-used records.

This implementation removes the earlier state-changing GET and raw-token persistence findings. The remaining production concerns are rate limiting, deployment-specific link origin, secret rotation, mail-delivery monitoring, explicit handling of expired links, and protected audit review.

Audit records are written directly through AuditLogRepository in major services. An audit entry can contain actor type/ID, action, target, result, source channel, and JSON metadata. Examples include CV failure, application submission/withdrawal, candidate invitation, status update, feedback, and auto-apply. Direct repository calls are simple, but a centralized audit facade could standardize metadata, request IP/user-agent capture, redaction, and failure behavior.

Secure email-action implementation



CareerFit IT AutoPilot — thesis diagram

Figure 3.9. Hashed email-action token and confirm-then-POST flow

3.10 Persistence and API Implementation

JPA entities map the domain tables, and Spring Data repositories provide pagination, sorting, locking, and explicit update operations. Flyway migrations V1–V24 create and update the current schema, including removal of passwordless login, CV review/draft states, the skill catalogue, Potential support, recruiter bookmarks, Job popularity, urgent hiring, account-level automation fields, and application deadlines. Hibernate uses `ddl-auto=validate`.

Database constraints are treated as correctness boundaries rather than only documentation. Unique indexes protect email identity, default CV, Matching, Application, and imported Job hash. Check constraints restrict roles, labels, statuses, source channels, and salary modes. Version columns support optimistic concurrency on mutable core entities. JSONB stores vectors and variable metadata, while relational foreign keys preserve ownership and lifecycle relationships.

Controllers return shared API envelopes and DTOs instead of exposing entities. Validation annotations reject malformed requests near the API boundary, while service exceptions represent not found, forbidden, bad request, and conflict cases. OpenAPI is generated through springdoc. Public list endpoints and authenticated workspaces use pagination or bounded top-result queries to avoid unbounded payloads.

Table 3.5. Representative API-to-service mapping

API operation	Controller/service responsibility	Persistence effect
---------------	-----------------------------------	--------------------

API operation	Controller/service responsibility	Persistence effect
POST /api/cv/upload; GET/PATCH /api/cv/{id}/review	Extract content, expose/edit review sections	CV, review data, quality signals, audit
POST /api/cv/{id}/review/confirm	Confirm owned CV and start vectorization/matching	CV vector, Matchings, audit
GET /api/matches/me/cards	Filter, sort, paginate, and map Candidate match cards	Read-only
POST /api/matches/{id}/feedback	Upsert feedback and trigger Rocchio after commit	Feedback, audit, learned Job vector, stale flags
POST /api/jobs/drafts; POST /api/jobs/{id}/publish	Validate company/JD/deadline and manage owned Job lifecycle	Job, vectors, Matchings, audit
PUT /api/recruiter/talent/jobs/{jobId}/candidates/{candidateId}/bookmark	Validate Job ownership and save Talent Pool choice	Recruiter CV bookmark
PATCH /api/automation/policy; POST /api/automation/pause	Update or pause the account's automation policy	Automation policy and version

3.11 Frontend Integration

App.tsx defines public, Candidate, Recruiter, and Administrator routes. Protected routes check the loaded account role, while session restoration calls /api/auth/me and clears invalid state. Public pages include Jobs and employer details. Candidate pages cover CV drafts and review, matching, urgent Jobs, recommendations, applications, analytics, AutoFit, and settings. Recruiter pages cover company onboarding, Jobs, applicants, Talent Pool, bookmarks, invitations, and analytics.

Frontend/src/lib/api.ts centralizes HTTP access. It reads VITE_API_BASE_URL with /api as the default, attaches the bearer token from sessionStorage, selects JSON headers except for multipart FormData, unwraps the shared response envelope, and throws a normalized error for unsuccessful responses. DTO mapping functions convert backend shapes and labels into UI models. React Query hooks manage loading, refetching, caching, polling, and error state for server data.